

News & Social Media Analytics Engine

Keyword, hashtag, sentiment and trend analysis using Data Structures and Algorithms (Java)

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Jhana Sri (2520090121)
Sutte Alekhya (2520090126)

Team Number: 02

Section: 12

Under the guidance of

Ms. K. Chandusha

Assistant Professor, CS&IT



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY



DECLARATION

The Mini Project Report entitled “**News & Social Media Analytics Engine**” is a record of Bonafide work of **Jhana Sri – 2520090121**, **Sutte Alekhya – 2520090126** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Jhana Sri – 2520090121
Sutte Alekhya – 2520090126

K L (Deemed to be) University

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY**



CERTIFICATE

This is to certify that the mini project based report entitled “**News & Social Media Analytics Engine**” is a bonafide work done and submitted by **Jhana Sri – 2520090121, Sutte Alekhya – 2520090126** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Supervisor

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our guide, Ms. K. Chandusha, Assistant Professor, CS&IT, which cannot be expressed in words, for the tremendous support, encouragement and guidance given for this project.

We express our gratitude to **Dr. Khaja Shareef**, Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, ways and means by which we were able to complete this project.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

Jhana Sri – 2520090121

Sutte Alekhya – 2520090126

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	16
8.	Conclusion & Future Scope	18
9.	References	19

1.ABSTRACT

The News & Social Media Analytics Engine is a Java-based application that analyses a collection of news articles and social-media posts and turns the raw text into organised information. With the rapid growth of digital platforms it is difficult to see which topics are popular and which content is most relevant, so the system reads posts from a CSV file, cleans the text by removing special characters and stop words, and then performs four analyses: keyword extraction, hashtag analysis, sentiment classification (positive, negative or neutral) and trending-topic detection. The project applies Data Structures and Algorithms to a real-world problem: HashMap is used for frequency counting, ArrayList for storing and managing posts, sorting and searching (linear and binary search) for ranking and retrieval, and PriorityQueue (min-heap) for finding the Top-K keywords, hashtags and trends in $O(n \log k)$ time. The engine was tested on a 30-post sample dataset. It identified #technology, #sports and #economy as the leading hashtags, classified 46.7% of posts as positive, 13.3% as neutral and 40.0% as negative, and ranked trending topics by combining frequency with likes. The results show that choosing the right data structure makes analysis of growing textual data efficient and easy to understand.

Team Name: Team 02

Team Logo (if any): —

Team Members:

1. Jhana Sri, 2520090121
2. Sutte Alekhya, 2520090126

2. INTRODUCTION

Background. The News & Social Media Analytics Engine is a Java-based application developed to analyse large amounts of news articles and social-media posts in an organised and efficient manner. With the rapid growth of digital platforms, users receive information from many sources, which makes it difficult to identify which topics are popular and which information is most relevant. The proposed system processes textual data and extracts useful information such as frequent keywords, hashtags, trending topics and sentiment patterns.

Motivation and approach. The main focus of the project is the practical application of Data Structures and Algorithms to a real-world analytics problem. A HashMap is used for efficient frequency counting, Lists for storing and managing data, searching and sorting algorithms for retrieving and ranking information, and a Priority Queue for finding the most important items. The engine reads its input from CSV/TXT files and works on a 30-post sample dataset so that every result can be checked by hand.

Organisation of the report. Section 3 reviews related work; Section 4 states the problem and objectives; Section 5 lists the hardware and software used; Section 6 presents the methodology and the code of the four main modules; Section 7 shows the results with discussion; and Section 8 concludes with future scope, followed by the references.

3. LITERATURE SURVEY

1. Apache Storm – A distributed real-time stream-processing framework, written for the Java Virtual Machine, in which spouts and bolts process unbounded streams of data. It is widely used for live analytics such as counting hashtags in a stream. Our project applies the same frequency-counting idea on a single machine using a HashMap.

2. Apache Kafka – A distributed log used to ingest and replay high-volume event streams such as posts, clicks and logs. It decouples the producers of data from the analytics programs that consume it. In our project the same role is played, on a small scale, by the CSV file that feeds posts to the engine.

3. Elasticsearch / Apache Lucene – A search engine, written in Java, that builds an inverted index mapping every term to the documents containing it. It supports fast full-text and keyword queries over very large collections. Our engine offers keyword search through linear search over posts and binary search over a sorted keyword list.

4. NetworkX – A Python library for analysing graphs and social networks, including shortest paths and PageRank. It is commonly used to find influential users in a network. It is a possible future extension of our project, which currently focuses on text rather than on user connections.

5. VADER Sentiment Analysis – A lexicon and rule-based sentiment tool designed for social-media text, introduced by Hutto and Gilbert. Each word carries a polarity score and the scores are combined to label a post. Our classifier follows the same lexicon principle with small positive and negative word sets stored in HashSets.

6. GDELT Project – A large open database that monitors news media worldwide and extracts themes, people, locations and tone from articles. It shows the value of analysing news at scale and over time. Our project applies a much smaller version of the same idea to keywords, hashtags and sentiment.

7. Google Trends – A public service that reports how often topics are searched and which topics are rising. It is a familiar example of trending-topic detection. Our engine provides a comparable feature by ranking topics with a score that combines frequency and likes, using a PriorityQueue.

8. Brandwatch – A commercial social-listening platform that tracks mentions of brands and topics across social networks and news sites, with sentiment reports. It shows the practical demand for such tools, but its algorithms are closed. Our engine reproduces the main features openly so that each step can be studied.

9. Sprout Social – A commercial social-media management suite that provides engagement analytics such as likes and shares per post and highlights the best-performing content. We use a similar idea to rank posts by likes through sorting.

Comparison of existing approaches. The table below compares the kinds of existing systems with the proposed system.

S.No	Existing approach	Technology / method	Limitations
1	News aggregation systems	Web APIs, web scraping	Mainly collect and display news
2	Keyword analysis systems	Hashing, frequency counting	Limited trend analysis
3	Sentiment analysis systems	Text processing	Often focus only on sentiment
4	Trending topic systems	Sorting, ranking	May not combine multiple analytics
5	Social media analytics	Hashtag and keyword analysis	Can be difficult to process large datasets
6	Proposed system	Java + DSA	Combines keyword, hashtag, sentiment and trend analysis

4. Problem Statement and Objectives

Problem Statement: To design a Java application that applies Data Structures and Algorithms to analyse news and social-media posts and report the popular keywords, hashtags, sentiment and trending topics.

Problem Description: Users receive far more information than they can read, and it is hard to tell which topics are popular; efficient data structures are needed to count, search and rank this text quickly.

Objectives:

Purpose of the project: to solve the problem of information overload in news and social media by building each analysis feature on a suitable data structure.

The objectives are:

- To read news and social-media data from CSV/TXT files and store it in an ArrayList of Post objects.
- To clean the text by removing special characters and stop words (HashSet look-up).
- To count keywords and hashtags efficiently using a HashMap ($O(1)$ average per update).
- To classify every post as Positive, Negative or Neutral using word lists.
- To find the Top-K keywords, hashtags and trending topics using a PriorityQueue (min-heap) in $O(n \log k)$.
- To rank posts by likes using sorting and to retrieve information using linear and binary search.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i3 or above – runs the application
- RAM: 4 GB minimum – supports program execution
- Storage: 10 GB free space – stores project files and data
- Monitor: 14" or above – displays the application
- Keyboard and Mouse: standard – user interaction

Software Requirements:

- Java – main programming language
- JDK 17 or later – compile and run the project
- IntelliJ IDEA / Eclipse – development environment
- Java Collections Framework – implements data structures and algorithms
- CSV / TXT files – store and read news and social-media data
- Git / GitHub – version control and code management

6. IMPLEMENTATION

Methodology. The workflow in Figure 6.1 is followed. Data is read from a TXT/CSV file, pre-processed (stop words and special characters removed) and analysed in four parallel ways – keyword extraction, hashtag analysis, sentiment analysis and trend detection – after which the results are stored and displayed. The system is implemented in four modules whose complete source code follows.

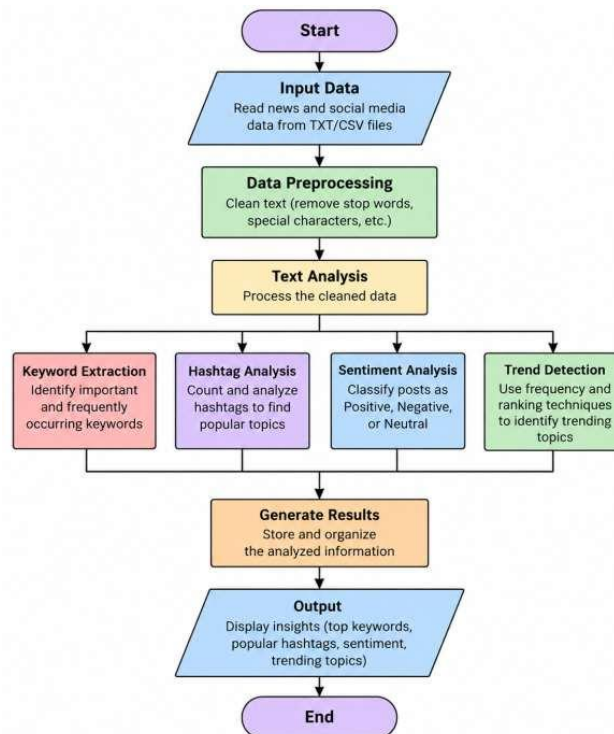


Figure 6.1: Methodology / workflow of the engine

Module 1: Post.java and DataLoader.java – reading the CSV data

```
/** A single news article or social-media post read from the CSV file. */
public class Post {
    final int id;
    final String source; // "News" or "Social"
    final int likes;
    final String text;
    String sentiment; // Positive / Negative / Neutral (filled by Analyzer)

    Post(int id, String source, int likes, String text) {
        this.id = id; this.source = source; this.likes = likes; this.text = text;
    }
    public String toString() {
        return "#" + id + " [" + source + "] likes=" + likes + " | " + text;
    }
}

import java.io.*;
```

```
import java.util.*;

/** Module 1: reads the CSV / TXT data file into an ArrayList of Post objects. */
public class DataLoader {
    public static List<Post> load(String path) throws IOException {
        List<Post> posts = new ArrayList<>();
        try (BufferedReader br = new BufferedReader(new FileReader(path))) {
            String line = br.readLine(); // skip header row
            while ((line = br.readLine()) != null) {
                if (line.trim().isEmpty()) continue;
                String[] p = line.split(",", 4); // id,source,likes,text
                posts.add(new Post(Integer.parseInt(p[0]), p[1],
                                    Integer.parseInt(p[2]), p[3]));
            }
        }
        return posts;
    }
}
```

Module 2: TextPreprocessor.java – cleaning the text

```
import java.util.*;

/** Module 2: cleans text - lower-case, removes special characters and stop words. */
public class TextPreprocessor {
    private static final Set<String> STOP = new HashSet<>(Arrays.asList(
        "the", "a", "an", "is", "are", "was", "to", "of", "and", "in", "on", "for", "as", "at", "by", "it",
        "this", "that", "with", "from", "after", "again", "new", "our", "we", "if", "do", "not", "but",
        "so", "be", "has", "have", "all", "every", "during", "ahead", "today", "week", "year", "across"));

    /** Extracts hashtags such as #cricket before special characters are removed. */
    public static List<String> hashtags(String text) {
        List<String> tags = new ArrayList<>();
        for (String w : text.split("\\s+"))
            if (w.startsWith("#") && w.length() > 1) tags.add(w.toLowerCase());
        return tags;
    }

    /** Returns the cleaned list of meaningful words (no hashtags, no stop words). */
    public static List<String> clean(String text) {
        List<String> words = new ArrayList<>();
        for (String w : text.split("\\s+")) {
            if (w.startsWith("#")) continue;
            w = w.toLowerCase().replaceAll("[^a-z]", ""); // strip special characters
            if (w.length() > 2 && !STOP.contains(w)) words.add(w);
        }
        return words;
    }
}
```

Module 3: Analyzer.java – keyword, hashtag, sentiment and trend analysis, sorting and searching

```
import java.util.*;

/** Module 3: keyword, hashtag, sentiment and trend analysis plus searching / sorting. */
public class Analyzer {
    private static final Set<String> POS = new HashSet<>(Arrays.asList(
        "great", "amazing", "excellent", "happy", "win", "success", "love", "loved", "good", "best",
        "proud", "congratulations", "boost", "growth", "record", "better", "faster", "strong"));
    private static final Set<String> NEG = new HashSet<>(Arrays.asList(
        "bad", "terrible", "worst", "angry", "sad", "loss", "crisis", "risk", "failed", "attack",
        "crashed", "delayed", "disappointed", "burden", "outage", "flood"));

    final List<Post> posts;
    final Map<String, Integer> keywordFreq = new HashMap<>(); // HashMap: frequency counting
    final Map<String, Integer> hashtagFreq = new HashMap<>();
    final Map<String, Double> trendScore = new HashMap<>();
    final Map<String, Integer> sentimentCount = new LinkedHashMap<>();

    Analyzer(List<Post> posts) {
```

```

        this.posts = posts;
        sentimentCount.put("Positive", 0); sentimentCount.put("Neutral", 0);
        sentimentCount.put("Negative", 0);
        for (Post p : posts) {
            List<String> words = TextPreprocessor.clean(p.text);
            for (String w : words) {
                keywordFreq.merge(w, 1, Integer::sum);
                trendScore.merge(w, 1 + p.likes / 100.0, Double::sum); // engagement-weighted
            }
            for (String t : TextPreprocessor.hashtags(p.text)) {
                hashtagFreq.merge(t, 1, Integer::sum);
                trendScore.merge(t, 1 + p.likes / 100.0, Double::sum);
            }
            p.sentiment = classify(words);
            sentimentCount.merge(p.sentiment, 1, Integer::sum);
        }
    }

    private String classify(List<String> words) {
        int s = 0;
        for (String w : words) { if (POS.contains(w)) s++; if (NEG.contains(w)) s--; }
        return s > 0 ? "Positive" : s < 0 ? "Negative" : "Neutral";
    }

    /** Top-K entries using a PriorityQueue (min-heap of size k): O(n log k). */
    static <T extends Number> List<Map.Entry<String,T>> topK(Map<String,T> map, int k) {
        PriorityQueue<Map.Entry<String,T>> pq = new PriorityQueue<>() {
            (a, b) -> Double.compare(a.getValue().doubleValue(), b.getValue().doubleValue());
        };
        for (Map.Entry<String,T> e : map.entrySet()) {
            pq.offer(e);
            if (pq.size() > k) pq.poll(); // drop the smallest
        }
        List<Map.Entry<String,T>> out = new ArrayList<>(pq);
        out.sort((a, b) -> Double.compare(b.getValue().doubleValue(),
a.getValue().doubleValue()));
        return out;
    }

    /** Sorting: posts ranked by likes, highest first. */
    List<Post> rankByLikes() {
        List<Post> sorted = new ArrayList<>(posts);
        sorted.sort((a, b) -> b.likes - a.likes);
        return sorted;
    }

    /** Linear search: all posts that contain the keyword. */
    List<Post> linearSearch(String keyword) {
        List<Post> found = new ArrayList<>();
        for (Post p : posts)
            if (p.text.toLowerCase().contains(keyword.toLowerCase())) found.add(p);
        return found;
    }

    /** Binary search on the alphabetically sorted keyword list: O(log n). */
    int binarySearch(List<String> sortedWords, String key) {
        int lo = 0, hi = sortedWords.size() - 1;
        while (lo <= hi) {
            int mid = (lo + hi) >>> 1, c = sortedWords.get(mid).compareTo(key);
            if (c == 0) return mid;
            if (c < 0) lo = mid + 1; else hi = mid - 1;
        }
        return -1;
    }
}

```

Module 4: Main.java – driver program

```

import java.util.*;

/** Module 4: driver - runs the full pipeline and prints the output. */
public class Main {
    public static void main(String[] args) throws Exception {
        List<Post> posts = DataLoader.load("posts.csv");
        Analyzer an = new Analyzer(posts);
    }
}

```

```

int news = 0; for (Post p : posts) if (p.source.equals("News")) news++;

System.out.println("=====");
System.out.println("    NEWS & SOCIAL MEDIA ANALYTICS ENGINE    (Java + DSA)");
System.out.println("=====");
System.out.println("Posts loaded: " + posts.size() + "    (News: " + news + ", Social: "
+ (posts.size() - news) + ")");
System.out.println("Distinct keywords: " + an.keywordFreq.size() + "    Distinct
hashtags: " + an.hashtagFreq.size());

System.out.println("\n[1] TOP 5 KEYWORDS (HashMap + PriorityQueue)");
int i = 1; for (Map.Entry<String,Integer> e : Analyzer.topK(an.keywordFreq, 5))
    System.out.printf("    %d. %-14s %d times\n", i++, e.getKey(), e.getValue());

System.out.println("\n[2] TOP 5 HASHTAGS (HashMap + PriorityQueue)");
i = 1; for (Map.Entry<String,Integer> e : Analyzer.topK(an.hashtagFreq, 5))
    System.out.printf("    %d. %-14s %d times\n", i++, e.getKey(), e.getValue());

System.out.println("\n[3] SENTIMENT ANALYSIS");
for (Map.Entry<String,Integer> e : an.sentimentCount.entrySet())
    System.out.printf("    %-9s %2d posts (%.1f%%)\n", e.getKey(), e.getValue(), 100.0 *
e.getValue() / posts.size());

System.out.println("\n[4] TRENDING TOPICS (frequency + likes, PriorityQueue ranking)");
i = 1; for (Map.Entry<String,Double> e : Analyzer.topK(an.trendScore, 5))
    System.out.printf("    %d. %-14s trend score %.2f\n", i++, e.getKey(),
e.getValue());

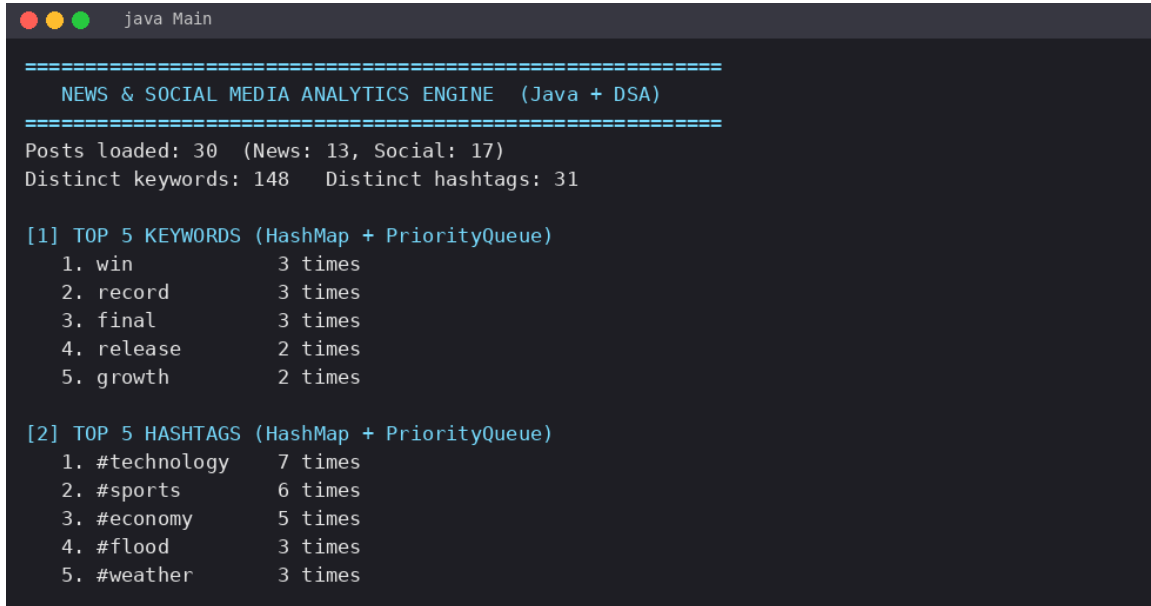
System.out.println("\n[5] TOP 3 POSTS BY LIKES (Sorting)");
List<Post> ranked = an.rankByLikes();
for (int k = 0; k < 3; k++) System.out.println("    " + ranked.get(k));

System.out.println("\n[6] SEARCH");
List<Post> found = an.linearSearch("cricket");
System.out.println("    Linear search 'cricket' -> " + found.size() + " posts: ids " +
ids(found));
List<String> sortedWords = new ArrayList<>(an.keywordFreq.keySet());
Collections.sort(sortedWords);
System.out.println("    Binary search 'technology' -> index " +
an.binarySearch(sortedWords, "technology") + " of " + sortedWords.size());
System.out.println("    Binary search 'football' -> index " +
an.binarySearch(sortedWords, "football") + " of " + sortedWords.size());
System.out.println("    Binary search 'zebra' -> index " +
an.binarySearch(sortedWords, "zebra") + " (not found)");
}
static String ids(List<Post> l) { List<Integer> r = new ArrayList<>(); for (Post p : l)
r.add(p.id); return r.toString(); }
}

```

7. RESULTS & DISCUSSION

The program was compiled with javac and run with java Main on the 30-post sample file posts.csv. The console output is shown in three screenshots.



```
java Main

=====
NEWS & SOCIAL MEDIA ANALYTICS ENGINE  (Java + DSA)
=====

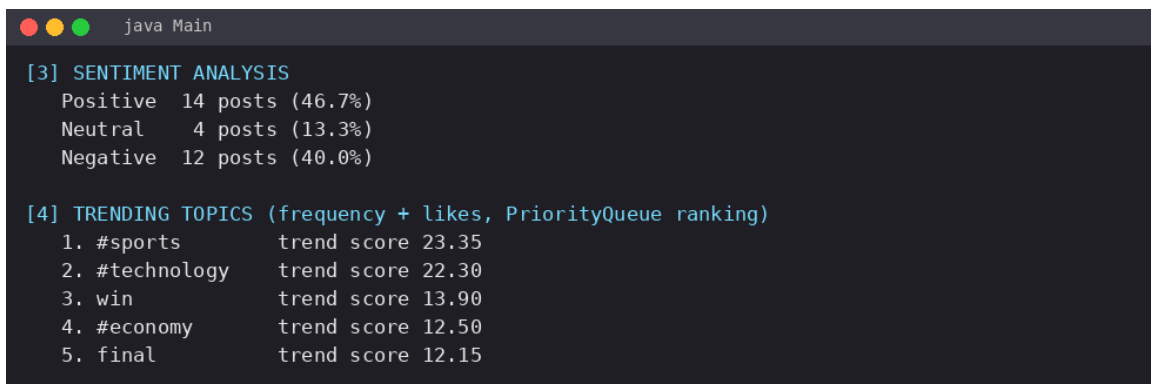
Posts loaded: 30  (News: 13, Social: 17)
Distinct keywords: 148  Distinct hashtags: 31

[1] TOP 5 KEYWORDS (HashMap + PriorityQueue)
1. win          3 times
2. record       3 times
3. final        3 times
4. release      2 times
5. growth       2 times

[2] TOP 5 HASHTAGS (HashMap + PriorityQueue)
1. #technology  7 times
2. #sports      6 times
3. #economy     5 times
4. #flood       3 times
5. #weather     3 times
```

Figure 7.1: Data summary, Top-5 keywords and Top-5 hashtags

The engine loaded 30 posts (13 news, 17 social) and found 148 distinct keywords and 31 hashtags. HashMap counts and a PriorityQueue give #technology (7), #sports (6) and #economy (5) as the most used hashtags.



```
java Main

[3] SENTIMENT ANALYSIS
Positive  14 posts (46.7%)
Neutral   4 posts (13.3%)
Negative  12 posts (40.0%)

[4] TRENDING TOPICS (frequency + likes, PriorityQueue ranking)
1. #sports      trend score 23.35
2. #technology  trend score 22.30
3. win          trend score 13.90
4. #economy     trend score 12.50
5. final        trend score 12.15
```

Figure 7.2: Sentiment analysis and trending topics

46.7% of posts were classified positive, 13.3% neutral and 40.0% negative. Trending topics combine frequency with likes, so #sports (23.35) and #technology (22.30) rank highest.

```
java Main

[5] TOP 3 POSTS BY LIKES (Sorting)
#6 [Social] likes=510 | Amazing performance by the team in the cricket final #cricket #sports
#25 [Social] likes=480 | AI is amazing but there is risk if we do not regulate it #AI #technology
#13 [Social] likes=420 | Great news the startup raised big funding to boost growth #startup #economy

[6] SEARCH
Linear search 'cricket' -> 2 posts: ids [2, 6]
Binary search 'technology' -> index 129 of 148
Binary search 'football' -> index 48 of 148
Binary search 'zebra' -> index -1 (not found)
```

Figure 7.3: Top posts by likes (sorting) and searching

Sorting ranks post #6 (510 likes) first. Linear search finds the keyword “cricket” in posts 2 and 6, and binary search locates “technology” and “football” in the sorted keyword list and correctly reports -1 for “zebra”.

Discussion. The table below explains why each data structure was chosen. The dataset is a small sample written for testing, so the numbers show that the algorithms work correctly and can be checked by hand; they are not statements about real-world news. Sentiment uses a small word list, so sentences with sarcasm or mixed feelings may be classified wrongly.

Feature	Data structure / algorithm	Time complexity
Keyword and hashtag counting	HashMap	$O(1)$ average per update
Stop-word and sentiment look-up	HashSet	$O(1)$ average
Top-K keywords / hashtags / trends	PriorityQueue (min-heap)	$O(n \log k)$
Ranking posts by likes	Sorting (List.sort)	$O(n \log n)$
Keyword search in posts	Linear search	$O(n)$
Search in sorted keyword list	Binary search	$O(\log n)$

8. CONCLUSION & FUTURE SCOPE

Conclusion. The News & Social Media Analytics Engine successfully demonstrates how Data Structures and Algorithms can be applied to analyse and organise continuously growing textual data. By processing news and social-media content, the system identifies popular keywords, hashtags, trending topics and sentiment, giving a clear overview of the information in the dataset. The use of HashMap, Lists, searching, sorting and PriorityQueue makes the system efficient for storing, retrieving, counting and ranking data, so all the objectives of the project were met.

Future scope.

Near term: replace the small word lists with a full sentiment lexicon, add stemming so that related words are counted together, and read data from larger CSV files.

Medium term: build a graphical or web dashboard with charts, add time-window trend detection using a queue, and store the data in a database.

Long term: connect the engine to live news and social-media APIs, scale it with a distributed framework such as Kafka and Spark, and add machine-learning models for topic detection and multilingual sentiment.

9. REFERENCES

1. Apache Storm – Distributed real-time computation system.
<https://storm.apache.org>
2. Apache Kafka – Distributed event streaming platform. <https://kafka.apache.org>
3. Elasticsearch – Distributed search and analytics engine (inverted index).
<https://www.elastic.co/elasticsearch>
4. NetworkX – Network analysis in Python. <https://networkx.org>
5. C. J. Hutto and E. Gilbert, “VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text,” ICWSM 2014.
<https://github.com/cjhutto/vaderSentiment>
6. The GDELT Project. <https://www.gdeltproject.org>
7. Google Trends. <https://trends.google.com>
8. Brandwatch – Social listening and analytics. <https://www.brandwatch.com>
9. Sprout Social – Social media analytics. <https://sproutsocial.com>
10. Oracle, Java Platform SE Documentation – Collections Framework (HashMap, ArrayList, PriorityQueue).
<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/package-summary.html>